

Ingestion & Ops — trading-bot

This README is a concise operations checklist and mapping for the repo's data ingestion pipelines. I

VM bootstrap checklist (minimal)

Run these steps on a VM (Ubuntu 22.04/24.04 recommended) to create a reproducible ingestion node.

1. Create a non-root user and update the system

```
sudo apt-get update && sudo apt-get upgrade -y
```

2. Install runtime tools

```
sudo apt-get install -y git curl build-essential unzip python3 python3-venv python3-pip
```

3. Clone repo and create Python environment

```
git clone <REPO_URL> trading-bot
cd trading-bot
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

4. Install system deps (optional via conda)

```
# Optional: if you prefer conda for binary wheel performance
# Install miniconda and create env then install pyarrow/fastparquet there
```

5. Install Python dependencies

```
pip install -r requirements.txt
pip install -r tradingbot_ibkr/requirements.txt
```

6. (If uploading to GCS) Configure GCP credentials

```
# Use either gcloud auth or a service account key
gcloud auth login
# OR
export GOOGLE_APPLICATION_CREDENTIALS=/path/to/sa-key.json
```

7. Ensure gsutil or google-cloud-storage is available

```
# If gsutil is available via gcloud, fine. Otherwise:
pip install google-cloud-storage
```

8. (Optional) Install `ccxt` for live fetches

```
pip install ccxt
```

9. Run the pilot pipeline (quick end-to-end test)

```
# Run the pipeline startup script which downloads small pilot symbols and uploads to GCS
bash tradingbot_ibkr/scripts/pipeline_startup.sh
```

```
# Or run steps individually (safer, inspect each step):
python tradingbot_ibkr/scripts/binance_download_all.py --since 2021-01 --until 2021-03 --symbols-
python tradingbot_ibkr/scripts/binance_vision_full_download.py --symbol BTCUSDT --since 2021-01 -
python scripts/binance_raw_to_parquet.py --raw /tmp/raw_all --out data/parquet/ohlcv_1m --symbols
```

Minimal local re-activation commands

- Backtest (CSV):

```
python scripts/run_backtest.py --source csv --path tradingbot_ibkr/datafiles/BTC_USDT_bars.csv --s
```

- Backtest (CCXT live fetch):

```
pip install ccxt
python scripts/run_backtest.py --source ccxt --exchange binance --symbol "BTC/USDT" --timeframe 1h
```

- Train model (LightGBM):

```
python scripts/train_lightgbm.py --source csv --path tradingbot_ibkr/datafiles/BTC_USDT_bars.csv -
```

- Convert raw CSV.GZ -> Parquet

```
BINANCE_RAW_DIR=/path/to/raw BINANCE_RAW_DIR_PARQUET_OUT=data/parquet/test python scrip
```

Ingestion map (script -> inputs -> outputs)

- `tradingbot\_ibkr/scripts/binance\_vision\_full\_download.py`
 - Inputs: remote archives at <https://data.binance.vision>
 - Outputs: local raw CSVs or merged CSV/Parquet in ``
 - Activate: `python tradingbot_ibkr/scripts/binance_vision_full_download.py --symbol BTCUSDT --sin
- `tradingbot\_ibkr/scripts/binance\_download\_all.py`
 - Inputs: ccxt symbol discovery or manual symbol list
 - Outputs: orchestrates per-symbol downloads (calls binance_vision_full_download)
- `tradingbot\_ibkr/binance\_vision\_to\_gcs.py`
 - Inputs: local raw or merged files
 - Outputs: uploaded to GCS bucket
 - Activate: `python tradingbot_ibkr/binance_vision_to_gcs.py --bucket my-bucket --symbol BTCUSDT -
- `tradingbot\_ibkr/binance\_trade\_dump\_ingest.py`
 - Inputs: Binance trade dump files
 - Outputs: normalized trade CSVs and optional OHLCV
- `scripts/binance\_raw\_to\_parquet.py`
 - Inputs: BINANCE_RAW_DIR (local *.csv.gz)
 - Outputs: Partitioned Parquet under PARQUET_OUT (default `data/parquet/ohlcv\_1m`)
 - Activate: `BINANCE_RAW_DIR=data/raw/binance/spot PARQUET_OUT=data/parquet/test python scripts
- `scripts/run\_backtest.py`
 - Inputs: `--source csv` reads `tradingbot\_ibkr/datafiles/<SYMBOL>\_bars.csv`; `--source ccxt` fetch
 - Outputs: backtest JSON reports in repo root (backtest_*.json)
- `scripts/train\_lightgbm.py`
 - Inputs: OHLCV via CSV or CCXT
 - Outputs: model artifacts and reports under `tradingbot\_ibkr/model\_store/`

Troubleshooting & notes

- CCXT & exchange access:
 - If `ModuleNotFoundError: No module named 'ccxt'` -> `pip install ccxt`
 - If Binance returns HTTP 451 (restricted location), run on a VM in a supported region or use a di
- GCS uploads:
 - `gsutil` requires `gcloud` or `GOOGLE\_APPLICATION\_CREDENTIALS` pointing to a service account JS
- Large downloads:
 - The `pipeline\_startup.sh` is intended for a dedicated VM. It installs conda and downloads potent

Alternative exchanges for CCXT live fetch

If Binance is geo-blocked in your environment, switch to an alternative exchange supported by CCXT t

- `bybit` (spot & derivatives) — id `bybit`
- `kucoin` — id `kucoin`
- `kraken` — id `kraken`
- `coinbasepro` / `coinbase` — id `coinbasepro`/`coinbase`
- `okx` — id `okx`

Usage example (replace `--exchange binance`):

```
python scripts/run_backtest.py --source ccxt --exchange bybit --symbol "BTC/USDT" --timeframe 1h -
```

Notes on exchange choice:

- Check exchange rate limits and API key requirements for private endpoints (OHLCV is public on most)
- Some exchanges use different symbol formats (e.g., `BTC/USDT` vs `BTC/USDT:USDT`); use `tradingbot`

Ops checklist for safe promotion to live

- Ensure `ALLOW\_MODEL\_PROMOTE=true` in environment and create `allow\_live\_confirm.txt` in repo root
- Run backtests and review backtest_*.json outputs before applying model to live.
- Use `tradingbot\_ibkr/worker.py` to manage jobs and `tradingbot\_ibkr/model\_store/jobs/` to queue pr

If you want, I can also:

- Create a runnable `bootstrap\_vm.sh` that automates the VM bootstrap steps above (I can add a conse
- Produce a compact PDF exported from this Markdown.
- Run a small CCXT backtest against an alternative exchange here (I can install any exchange-specifi

Which follow-up should I do now?